

Introducción

Vistas na arquitectura ANSI/SPARC

- Permiten a definición dos diferentes esquemas externos.
- Ofrecen a cada usuario ou rol a visión da BD adecuada.
- Axudan a conseguir a independencia lóxica.

As vistas para o SQL e os SXBD relacionais

- Unha vista é unha "táboa virtual" que expón os datos resultando dunha consulta sobre outras táboas ou vistas.
- Táboa: Almacena a definición no catálogo, e os datos no espazo de datos.
 - Táboas permanentes: Manteñen a estrutura e os datos, que poden ser accedidos por outros usuarios ou sesións (control de permisos).
 - Táboas temporais: Manteñen a estrutura pero os datos elimínanse (ó acabar cada transacción ou a sesión que os creou). Os datos son privados a cada sesión.
- Vista: Só almacena a definición no catálogo. Non almacena datos. Componentes:
 - Nome, único no espazo de nomes de táboas e vistas.
 - Lista de atributos ou columnas.
 - Definición: sentencia `SELECT` que obtén os datos.
- Vista materializada: Caso especial de Vista na que si se almacenan os datos (as filas) no espazo de datos.

Vistas no SQL estándar

Creación de vistas

Definidas según o SQL-92:

```
CREATE VIEW nome_vista [( <lista_atributos> )]  
    AS <sentencia_select>  
    [ <check_option> ]
```

Consideracións:

- O esquema (lista de atributos explícito ou implícito debe ser válido.
- Un `select *` expándese á lista de atributos.
- A sentencia `select` non incluírá `ordet by` (SQL:2008 si o permite).

- A `<check_option>` só se aplica se a vista é actualizable.

Exemplos:

```
create view emp10
  as select *
    from emp
   where deptno = 10;

create view emp10_sal
  as select empno, ename, sal+coalesce(comm, 0) as sal_total
    from emp 10;

create view resumedep(deptno, nome dep, numemps, salmedio)
  as select deptno, dname, count(*), avg(sal)
    from emp natural join dept
   group by deptno, dname;
```

Eliminación e modificación de vistas

Eliminación de vistas

- `DROP VIEW nome_vista [ RESTRICT | CASCADE ]`
- Consideracións:
 - `restrict` (predeterminado): Se hai vistas dependentes, non se borra.
 - `cascade`: Borra as vistas dependentes e logo a actual.
- Exemplos

```
drop view emp10; -- falla: emp10_sal depende de emp10
drop view emp10 cascade; -- elimina emp10_sal e logo emp10
drop view resumedep restrict; -- Ok
```

Modificación de vistas

Non existe a sentencia `alter view` para modificar unha vista (sería necesario `drop` e logo `create` coa nova definición).

Actualización de vistas

Unha sentencia DML sobre unha vista trasladarase sobre as táboas base sobre as que se define a vista.

Regla (informal) básica: Para actualizar unha vista, o SXBD debe poder chegar dunha fila da vista a unha única fila da táboa base.

Normas (SQL-92):

Unha vista será actualizable se a consulta que a define cumple todas as condicións seguintes

- Non hai eliminación de duplicados nin agrupamento: Non se usa `select distinct` nin as cláusulas `group by` ou `having`.
- Non hai máis dunha táboa no `from` (a vista non se define con joins).
- A consulta non é o resultado de operacións alxebraicas (`union`, `intersec`, `except`).
- Se hai cláusula `where`, esta non pode conter unha subconsulta que use a mesma táboa que se usa na cláusula `from`.

Ademáis:

- Deben satisfacer as restriccións da táboa base. En xeral, se omitimos un atributo coa restricción `not null`, a vista non permitirá insercións.
- Se a vista utiliza expresións, a vista non permitirá actualizar (`update`) a expresión, nin a inserción de novas filas.

Extensións (SQL:1999 e posteriores): Cada revisión do estándar SQL inclúe capacidades opcionais para a actualizaicón de vistas. Por exeplo:

- Se a vista está definida sobre un `join`, os atributos da táboa preservada pola clave primaria serán actualizables.

Actualización de vistas: `check option`

Tuplas migratorias:

Unha *tupla migratoria* é aquela que "desaparece" da vista

- Insertamos unha fila que non cumple as condicións da vista: insértase na táboa base e non se ve na vista.
- Modificamos unha fila facendo que non cumpla as condicións: modifícase na táboa base e desaparece da vista.

```
insert into emp10(empno, ename, deptno)
  values(1234, 'PEPE', 20);

update emp10
  set deptno=20
  where ename='CLARK';
```

Check option

- Inclúese na sentencia de creación da vista, despois da definición da consulta `WITH [ CASCADED | LOCAL | CHECK OPTION ]`
- Permite evitar a aparición de tuplas migratorias.
- Só se admite en vistas actualizables.
- Opción predeterminada: `cascaded`

```
create view emp10check
  as select * from emp
    where deptno = 10;
  with check option;

insert into emp10check(empno, ename, deptno) --falla
  values(1234, 'PEPE', 20);
```

Alcance da comprobación: Diferencias entre `cascaded` e `local`: só cando as vistas están definida sobre outra vista.

- `local`: Comproba a condición da propia vista.
- `cascaded`: Comproba a condición da propia vista e de todas aquelas sobre as que está definida.

```
create view clerks10loc
  as select empno, ename, job, deptno
    from emp10
    where job='CLERK'
    with LOCAL check option;

-- Non inserta
insert into clerks10loc
  values(1234, 'PEPE', 'MANAGER', 10);

-- ???
insert into clerks10loc
  values(1234, 'PEPE', 'CLERK', 20);

create view clerks10casc
  as select empno, ename, job, deptno
    from emp10
    where job = 'CLERK'
    with CASCADED check option;

-- Non inserta
```

```
insert into clecrks10casc
  values(1234, 'PEPE', 'MANAGER', 10);

-- ???
insert into clecrks10casc
  values(1234, 'PEPE', 'CLERK', 20);
```

Ventajas e inconvenientes

Ventajas do uso das vistas

- Permiten a definición dos diferentes esquemas externos.
- Axudan a conseguir a independencia lóxica (dentro do posible, absorbe modificacións ou reestruturación de táboas).
- Facilita a realización de consultas complexas.
- Permite establecer condicións de seguridade, por exemplo ocultando datos.
- Permite establecer condicións de integridade de datos (usando `check option`).
- Os datos sempre están actualizados (con respecto ás táboas base).

Desventajas ou limitacións das vistas

- Limitacións á hora de actualizar datos a través das vistas.
- Non aumentan a eficiencia das consultas.

Cando se lanza unha consulta sobre unha vista, o SQL da consulta mézclase co SQL da vista (reescritura de consultas, *query rewriting*).

Non se "executa", a definición da vista e queda unha "cache" dese resultado para poder usarse por varias consultas sobre a vista.

O uso da vista tampouco diminúe a eficiencia da consulta (a reescritura ten lugar na fase de optimización).

Vistas en Oracle

Algún aspecto específico de Oracle

Creación de vistas

```
CREATE [OR REPLACE] VIEW <nome_vista> [(<lista_atributos>)]
AS <sentencia_select>
[WITH {READ ONLY | CHECK OPTION} [CONSTRAINT <nome_restrinción> ] ]
```

- Sigue fundamentalmente o estándar na parte relacional.

- Ten moitas extensións (obxectos, XML, JSON, ...).
- Permite `order by` na sentencia `select`.
- Permite vistas de só lectura (`with read only`).
- A cláusula `with check option` non ten especificación de alcance, sempre actúa en cascada.
- Pode especificar outras restricións (PK, FK) limitadas para as vistas.
- Usa `CREATE OR REPLACE VIEW ...` para modificar a definición dunha vista.

Borrado de vistas

```
DROP VIEW nome_vista;
```

- Borra a vista e marca como inválidas as vistas definidas sobre ela.
- Pode incluír a cláusula `cascade constraints` se a vista tivese restricións.

Alter de vistas

```
ALTER VIEW nome_vista COMPILE
```

- Utilizado cando unha vista deixou de ser válida, comproba dependencias e pode volver a marcala como válida
- Outras opcións de `alter view` sirven para engadir ou sacar restricións, ou poñer a vista en modo de lectura/escritura ou só lectura.
- `Alter view` non sirve para modificar a consulta asociada á vista (debe usarse `create or replace view`).

Actualizacións a través das visas

Siguen en certa forma o estándar, pero ...

- Permiten insertar filas cando a vista ten expresións (omitindo a expresión).
- Ignora a regra da subconsulta no where que referencia a mesma táboa sobre a que se define a vista (con resultados extraños).
- Permite actualizar vistas definidas sobre joins (só a táboa preservada por clave).

```
insert into emp10_sal values(1234, 'pepe', null);
* ORA-01733: virtual column not allower here

insert into emp10_sal(empno, ename) values(1234, 'pepe'); -- Inserta.

create view empcaro
  as select * from emp
     where sal > (select avg(sal) from emp);

delete from empcaro;
```

```
create view empdep
as select empno, ename, sal, e.deptno, dname
   from emp e join dept d on e.deptno = d.deptno;

insert into empdep(empno, ename, sal, deptno) values(1234, 'pepe', 1000,
10);
```

Vistas Materializadas en Oracle

- Tamén coñecidas como *snapshots*
- Almacenan o resultado da consulta en disco.
 - As consultas son máis rápidas.
 - Ocupan espacio en disco.
 - Hai que actualizar o contenido.
- Apareceron para tratar de aumentar o rendemento cando se consulta sobre vistas.
 - Especialmente en entornos distribuídos, data warehouses, ...
- Oracle permite especificar **cando materializar** a vista (executar a sentencia e gardar os datos en disco), e **como e cando actualizar** (refrescar) eses datos para mentelos sincronizados coas táboas base.

Materialización (build)

- Inmediata (`immediate`): materialízase cando se define (cando se lanza a sentencia `create materialized view`).
- Aplazada (`deferred`): A consulta executarase a posteriori (no primeiro *refresco* ou actualización).
- Utilizando unha táboa preexistente como datos actuais da vista (`on prebuilt table`).
 - Fai que a táboa e a vista materializada compartan o mesmo segmento de datos.
 - Úsase especialmente en data warehousing; non estará sincronizada, e ten limitacións.

Sincronizar o contido da vista coas táboas base

Como se actualiza o contido da vista materializada (modo de `refresh`)

- Recreando a vista por completo (`refresh complete`).
 - Pode facerse en calquera momento.
 - Caro (tempo e esforzo computacional).
- De forma incremental, actualizando só a nova información (`refresh fast`).
 - Require "materialized view logs" nas táboas base. Non sempre é posible.
 - Máis rápido.

- `refresh force` (predeterminado) fai incremental se é posible, e se non fai o completo.

Cando se actualiza

- Durante o commit de cada transacción que afecta ás táboas base (`on commit`).
 - Maior sincronismo dos datos entre a vista e as táboas base.
 - Non se permite para todos os tipos de vista. Alonga o proceso de commit.
- Baixo demanda (`on demand`): O refresco iníciase de forma manual
 - Utiliza procedementos almacenados no paquete `dbms_mview`.
- De forma periódica. Ex. a medianoite todos os días (`start with sysdate next sysdate + 1`)
- Nunca (`never refresh`).

Exemplos

```
CREATE MATERIALIZED VIEW MV_EMPDEP
  BUILD IMMEDIATE -- Predeterminado
  REFRESH FORCE    -- Predeterminado
  ON DEMAND       -- Predeterminado
  ENABLE QUERY REWRITE
AS
SELECT *
  FROM EMP NATURAL JOIN DEPT;

CREATE MATERIALIZED VIEW MV_EMPHOURS
  BUILD DEFERRED
  REFRESH COMPLETE
  START WITH SYSDATE NEXT SYSDATE + 1
AS
SELECT E.EMPNO, ENAME, P.PRNO, PNAME, HOURS
  FROM EMP E
        JOIN EMPPRO EP ON E.EMPNO=EP.EMPNO
        JOIN PRO P ON EP.PRNO=P.PRNO;

select mviewname, build_mode build, refresh_method metodo_ref,
       refresh_mode modo_ref, fast_refreshable, rewrite_enabled rewrite
  from user_mviews;
```

MVIEW_NAME	BUILD	METODO_REF	MODULO_REF	FAST_REFRESHABLE	REWRITE
-----	-----	-----	-----	-----	-----
MV_EMPDEP	IMMEDIATE	FORCE	DEMAND	NO	Y
MV_EMPHOURS	DEFERRED	COMPLETE	DEMAND	NO	N

Selección de datos

- Poden facerse consultas directamente sobre as vistas materializadas como se fosen táboas.
- Poden engadirse índices ás vistas materializadas para facer máis eficientes algunhas consultas.

```
create index i_mv_empdep_sal
  on mv_empdep(sal);

select *
  from mv_empdep
 where sal > 3000;
```

Reescritura de consultas

O `query rewrite` ou reescritura de consultas permite que o optimizador de Oracle utilice de forma *intelixente* as vistas materializadas aínda que non se referencien na consulta.

Se a consulta, ou parte dela, coincide coa definición da vista, en algúns casos pode usar a vista en lugar das táboas.

Este comportamento pode activarse ou desactivarse mediante:

```
ALTER SESSION SET QUERY_REWRITE_ENABLED = { TRUE | FALSE };
```

```
select *
  from emp e
       join dept d on e.deptno=d.deptno
 where d.deptno > 20;
```